
GRAFIKA KOMPUTEROWA - PROJEKT

1

Table of Contents

Joanna Dagil 231008	1
OPIS PROJEKTU	1
CZESC 1	2
CZESC 1 (A) (nie do oceny - ponizej znajduje się tez czesc (b))	2
CZESC 1 (B)	6
CZESC 2 (B)	8
FUNKCJE POMOCNICZE	10
Funkcja tworzy binarną maskę trójkąta na obrazie.	10
Funkcja rysuje odcinek na obrazie monochromatycznym.	11
Funkcja wypełnia zamknięty obszar obrazu metoda floodfill.	12
Funkcja do zamiany współrzędnych matematycznych na pikselowe i odwrotnie	14
Funkcja do wyznaczania parametrów płaszczyzny $Ax+By+Cz+D=0$ przechodzącej przez trzy punkty $p1,p2,p3$...	14
Funkcja do wyznaczania odległości punktu $[x,y,z]$ od płaszczyzny o równaniu $Ax+By+Cz+D=0$	15
Funkcja tworzy macierz transformacji jednorodnej 4x4 dla obrotu i przesunięcia.	15

Joanna Dagil 231008

OPIS PROJEKTU

Celem projektu było wygenerowanie obrazu oraz animacji dwóch nieprzezroczystych trójkątów umieszczonych w przestrzeni 3D i obserwowanych przez kamerę perspektywiczną.

W rozwiązaniu wykorzystano współrzędne jednorodne, dzięki czemu punkty trójkątów mogą być łatwo przekształcane za pomocą macierzy. Wierzchołki trójkątów są najpierw rzutowane na płaszczyznę obrazu przy użyciu macierzy rzutu perspektywicznego. Po rzutowaniu współrzędne matematyczne są zamieniane na współrzędne pikselowe, a następnie tworzona jest maska każdego trójkąta. Maskę powstaje przez narysowanie trzech krawędzi trójkąta i wypełnienie jego wnętrza metodą floodfill.

Do usuwania powierzchni zasłoniętych zastosowano z-buffer. Dla każdego piksela sprawdzane jest, czy należy on do rzutu danego trójkąta. Jeśli tak, wyznaczana jest odległość od kamery do punktu przecięcia promienia widzenia z płaszczyzną trójkąta. W obrazie końcowym zapisywany jest kolor tego trójkąta, który znajduje się najbliżej kamery. Pozwala to poprawnie rozstrzygnąć, który obiekt jest widoczny w miejscach, gdzie rzuty trójkątów nachodzą na siebie.

Kolor widocznej strony trójkąta wyznaczany jest na podstawie orientacji jego powierzchni względem kamery. W tym celu obliczany jest wektor normalny trójkąta jako iloczyn wektorowy dwóch jego boków. Następnie sprawdzany jest znak iloczynu skalarnego normalnej z wektorem skierowanym od środka trójkąta do kamery. Na tej podstawie wybierany jest kolor jednej albo drugiej strony trójkąta.

W części z dziurą w trójkącie A utworzono dodatkową maskę odpowiadającą rzutowi trójkątnej dziury. Piksele należące do tej maski zostają usunięte z maski trójkąta A. Dzięki temu obszar dziury nie bierze udziału w z-bufferze jako powierzchnia trójkąta A i może być w tym miejscu widoczny trójkąt B albo czarne tło.

W czesci animacyjnej dla kazdej klatki wyznaczane jest nowe polozenie trojkatow. Do tego wykorzystano macierz transformacji RPYT, laczaca obroty roll, pitch, yaw oraz przesuniecie. Po wykonaniu transformacji kazda klatka jest generowana od poczatku: punkty sa rzutowane, tworzone sa maski trojkatow, wycinana jest dziura, wybierane sa widoczne kolory stron oraz wykonywany jest z-buffer. Gotowe klatki sa zapisywane do pliku wideo.

```
%clear; close all; clc;
```

CZESC 1

CZESC 1 (A) (nie do oceny - ponizej znajduje się tez czesc (b))

```
% Dane sa dwa nieprzezroczyste trojkaty A i B umieszczone w 3D przestrzeni XYZ.
```

```
% Trojkat A ma wierzcholki - zapisane od razu we wspolrzecznych jednorodnych
```

```
tri(1).A = [ 2; 0.5; 10; 1];  
tri(1).B = [ -1; -1.5; 12; 1];  
tri(1).C = [ -1; 1.5; 8; 1];
```

```
% i kolor czerwony z jednej strony, a zielony z drugiej
```

```
tri(1).front = [255, 0, 0];  
tri(1).back = [0, 255, 0];
```

```
% a trojkat B ma wierzcholki
```

```
tri(2).A = [ 0.5; -2; 10; 1];  
tri(2).B = [-0.5; 2; 11; 1];  
tri(2).C = [ 1.5; 2; 9; 1];
```

```
% i kolor niebieski z jednej strony, a zolty z drugiej
```

```
tri(2).front = [0, 0, 255];  
tri(2).back = [255, 255, 0];
```

```
% Trojkaty sa obserwowane przez kamere o ogniskowej
```

```
f = 5;
```

```
% umieszczona w punkcie (0,0,0) i skierowana wzdluz osi Z
```

```
camera = [0; 0; -f];
```

```
% macierz transformacji rzutu perspektywicznego
```

```
H = zeros(4,4); H(1,1)=1; H(2,2)=1; H(4,3)=-1/f; H(4,4)=1;
```

```
% Obraz otrzymany z kamery ma miec rozdzielczosc
```

```
X = 640;
```

```
Y = 480;
```

```
% A rozmiar pixela to
```

```
dx = 0.01;
```

```
dy = 0.01;
```

```
% Tlo ma kolor czarny
```

```
Im = uint8(zeros(Y, X, 3));
```

```
% z-buffer przechowujący odległości do najbliższego obiektu dla każdego
piksela
z_buffer = inf(Y,X);

% rzutowanie trojkata A
tri(1).A_proj = H * tri(1).A; tri(1).A_proj = tri(1).A_proj /
tri(1).A_proj(4);
tri(1).B_proj = H * tri(1).B; tri(1).B_proj = tri(1).B_proj /
tri(1).B_proj(4);
tri(1).C_proj = H * tri(1).C; tri(1).C_proj = tri(1).C_proj /
tri(1).C_proj(4);
% rzutowanie trojkata B
tri(2).A_proj = H * tri(2).A; tri(2).A_proj = tri(2).A_proj /
tri(2).A_proj(4);
tri(2).B_proj = H * tri(2).B; tri(2).B_proj = tri(2).B_proj /
tri(2).B_proj(4);
tri(2).C_proj = H * tri(2).C; tri(2).C_proj = tri(2).C_proj /
tri(2).C_proj(4);

% rysowanie trojkata A
tri(1).triangle = triangle(X, Y, tri(1).A_proj, tri(1).B_proj, tri(1).C_proj,
dx, dy, 255);
% rysowanie trojkata B
tri(2).triangle = triangle(X, Y, tri(2).A_proj, tri(2).B_proj, tri(2).C_proj,
dx, dy, 255);

% wyznaczanie płaszczyzny trojkata A
[aa, bb, cc, dd] = plane(tri(1).A, tri(1).B, tri(1).C); tri(1).plane = [aa,
bb, cc, dd];
% wyznaczanie płaszczyzny trojkata B
[aa, bb, cc, dd] = plane(tri(2).A, tri(2).B, tri(2).C); tri(2).plane = [aa,
bb, cc, dd];

camera = [0; 0; -f];

% wyznaczenie koloru widocznego
for k = 1:2
    v1 = tri(k).B(1:3) - tri(k).A(1:3);
    v2 = tri(k).C(1:3) - tri(k).A(1:3);

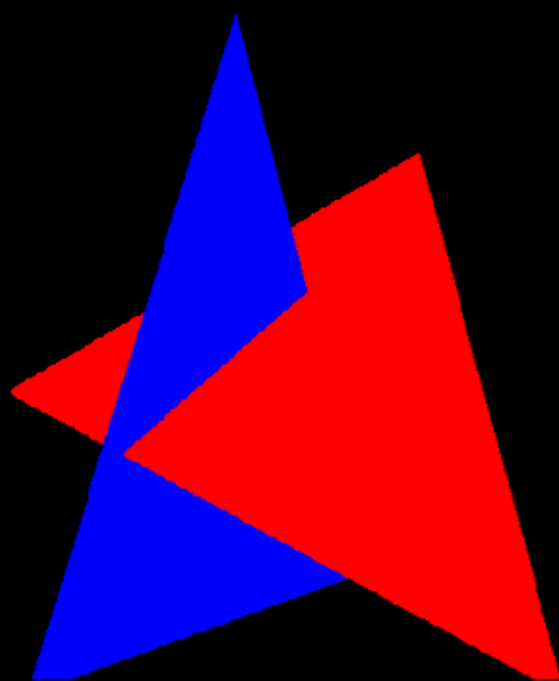
    center = (tri(k).A(1:3) + tri(k).B(1:3) + tri(k).C(1:3)) / 3;
    view_vec = camera - center;

    if dot(cross(v1, v2), view_vec) > 0
        tri(k).visible_color = tri(k).front;
    else
        tri(k).visible_color = tri(k).back;
    end
end
```

```
for j = 1:Y
    for i = 1:X
        [x,y] = pix_to_mat(i, j, X, Y, dx, dy);

        for k = 1:2
            if tri(k).triangle(j,i) > 0
                [d,~,~,~] = dist_to_plane(tri(k).plane, x, y, 0, -x, -y, f);
                if d < z_buffer(j, i)
                    z_buffer(j, i) = d;
                    Im(j, i, :) = tri(k).visible_color;
                end
            end
        end
    end
end

figure(1); imshow(Im);
```



CZESC 1 (B)

% W trojkacie A dodatkowo pojawila się trojkatna dziura zdefiniowana przez narozniki o wspolrzecznych:

```
tri(3).A = [0.8; 0.3; 10; 1];  
tri(3).B = [-0.25; -0.25; 10.5; 1];  
tri(3).C = [-0.25; 0.5; 9.5; 1];
```

% rzutowanie dziury

```
tri(3).A_proj = H * tri(3).A; tri(3).A_proj = tri(3).A_proj /  
tri(3).A_proj(4);  
tri(3).B_proj = H * tri(3).B; tri(3).B_proj = tri(3).B_proj /  
tri(3).B_proj(4);  
tri(3).C_proj = H * tri(3).C; tri(3).C_proj = tri(3).C_proj /  
tri(3).C_proj(4);
```

% rysowanie dziury

```
tri(3).triangle = triangle(X, Y, tri(3).A_proj, tri(3).B_proj, tri(3).C_proj,  
dx, dy, 255);
```

% wycinanie dziury z trojkata A

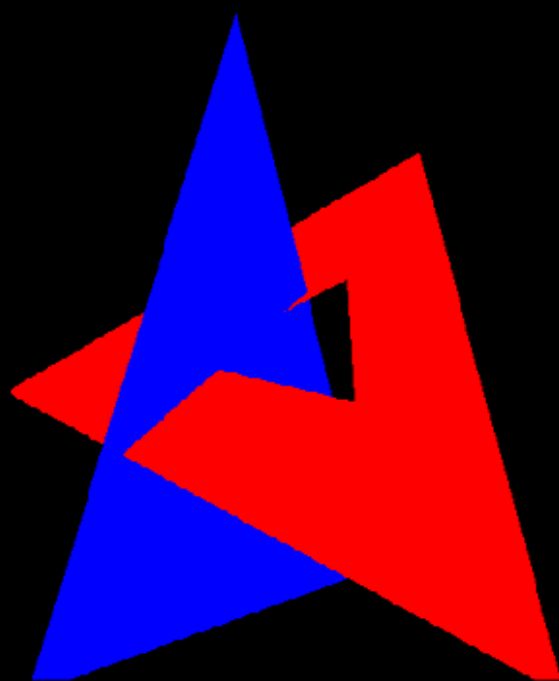
```
tri(1).triangle(tri(3).triangle > 0) = 0;
```

```
Im = uint8(zeros(Y, X, 3));
```

```
z_buffer = inf(Y,X);
```

```
for j = 1:Y  
    for i = 1:X  
        [x,y] = pix_to_mat(i, j, X, Y, dx, dy);  
  
        for k = 1:2  
            if tri(k).triangle(j,i) > 0  
                [d,~,~,~] = dist_to_plane(tri(k).plane, x, y, 0, -x, -y, f);  
                if d < z_buffer(j, i)  
                    z_buffer(j, i) = d;  
                    Im(j, i, :) = tri(k).visible_color;  
                end  
            end  
        end  
    end  
end  
end
```

```
figure(2); imshow(Im);
```



CZESC 2 (B)

% Dane sa dwa nieprzezroczyste trojkaty A i B umieszczone w 3D przestrzeni XYZ.

% Trojkat A ma wierzcholki - zapisane od razu we współrzędnych jednorodnych

```
tri(1).A = [ 2; 0; 0; 1];  
tri(1).B = [-1; -2; 2; 1];  
tri(1).C = [-1; 1; -2; 1];
```

% W trojkacie A dodatkowo pojawila się trojkatna dziura zdefiniowana przez narozniki o współrzędnych:

```
tri(3).A = [ 0.8; -0.2; 0; 1];  
tri(3).B = [-0.25; -0.75; 0.5; 1];  
tri(3).C = [-0.25; 0; -0.5; 1];
```

% a trojkat B ma wierzcholki

```
tri(2).A = [ 0; -2; 0; 1];  
tri(2).B = [-1; 2; 1; 1];  
tri(2).C = [ 1; 2; -1; 1];
```

% Zakladamy takie samo polozenie kamery oraz takie same parametry obrazow cyfrowych (klatek animacji) jak w czesci 1.

% Animacja wizualizująca opisany poniżej ruch obu trojkatow, zlozona z 180 klatek.

```
frames = 180;
```

% Przesuniecie trojkatow w stosunku do polozen poczatkowych opisane sa zawsze takimi samymi wektorami, tzn. dla trojkata A:

```
tri(1).move = [0, 0.5, 10];  
tri(3).move = tri(1).move; % tak samo dla dziury  
% dla trojkata B  
tri(2).move = [0.5, 0, 10];
```

% W kazdej klatce polozenie trojkata opisane jest obrotem (w stosunku do pozycji poczatkowej) oraz przesuniecie (tez w stosunku do pozycji poczatkowej). Obrot jest zlozeniem obrotu wokol osi OZ ukkladu współrzędnych (tzn. kat ROLL) oraz obrotu wokol osi OY ukkladu współrzędnych (tzn. kat PITCH).

% Dla trojkata A w n-tej klatce kat ROLL ma wartosc $n \times 2.a$ stopnia (gdzie a jest piata cyfra numeru albumu).

% Index 231008, więc

```
a = 0; b = 8;
```

```
tri(1).roll = 2.0 + a * 0.1;
```

```
tri(3).roll = tri(1).roll; % tak samo dla dziury
```

% Kat PITCH ma wartosc $n \times 1.b$ stopnia (gdzie b jest szosta cyfra numeru albumu).

```
tri(1).pitch = 1.0 + b * 0.1;
```

```
tri(3).pitch = tri(1).pitch; % tak samo dla dziury
```

% Dla trojkata B w n-tej klatce kat ROLL ma wartosc $n \times 2.b$ stopnia,


```
tri(2).roll = -(2.0 + b * 0.1);
% Kat PITCH ma wartosc -n*1.a stopnia.
tri(2).pitch = -(1.0 + a * 0.1);

%video = VideoWriter('triangles.avi');
video = VideoWriter('triangles_short.avi');
video.FrameRate = 20;
open(video);

% CZESC 2 SKROCONA
for n = 36:45
% CZESC 2 PELNA
%for n = 0:(frames-1)
    %fprintf('Generowanie klatki %d...\n', n);
    for k = 1:3
        % transformacje
        temp(k).H = RPYT(n*tri(k).roll, n*tri(k).pitch, 0, tri(k).move(1),
tri(k).move(2), tri(k).move(3));
        temp(k).A = temp(k).H * tri(k).A;
        temp(k).B = temp(k).H * tri(k).B;
        temp(k).C = temp(k).H * tri(k).C;

        temp(k).A_proj = H * temp(k).A;
        temp(k).A_proj = temp(k).A_proj / temp(k).A_proj(4);

        temp(k).B_proj = H * temp(k).B;
        temp(k).B_proj = temp(k).B_proj / temp(k).B_proj(4);

        temp(k).C_proj = H * temp(k).C;
        temp(k).C_proj = temp(k).C_proj / temp(k).C_proj(4);
        % rysowanie trojkata
        temp(k).triangle = triangle(X, Y, temp(k).A_proj, temp(k).B_proj,
temp(k).C_proj, dx, dy, 255);
    end

    % wycinanie dziury z trojkata A
    temp(1).triangle(temp(3).triangle > 0) = 0;

    for k = 1:2
        % wyznaczanie plaszczyzny trojkata
        [aa, bb, cc, dd] = plane(temp(k).A, temp(k).B, temp(k).C);
        temp(k).plane = [aa, bb, cc, dd];

        % wyznaczenie koloru widocznego
        v1 = temp(k).B(1:3) - temp(k).A(1:3);
        v2 = temp(k).C(1:3) - temp(k).A(1:3);
        center = (temp(k).A(1:3) + temp(k).B(1:3) + temp(k).C(1:3)) / 3;
        view_vec = camera - center;
        if dot(cross(v1, v2), view_vec) > 0
            temp(k).visible_color = tri(k).front;
        else
            temp(k).visible_color = tri(k).back;
        end
    end
end
```

```
Im = uint8(zeros(Y, X, 3));

z_buffer = inf(Y,X);

for j = 1:Y
    for i = 1:X
        [x,y] = pix_to_mat(i, j, X, Y, dx, dy);
        for k = 1:2
            if temp(k).triangle(j,i) > 0
                [d,~,~,~] = dist_to_plane(temp(k).plane, x, y, 0, -x, -y,
f);
                    if d < z_buffer(j, i)
                        z_buffer(j, i) = d;
                        Im(j, i, :) = temp(k).visible_color;
                    end
                end
            end
        end
    end

    writeVideo(video, Im);
end

close(video);
```

FUNKCJE POMOCNICZE

Funkcja tworzy binarną maskę trójkąta na obrazie.

Najpierw zamienia współrzędne matematyczne wierzchołków na pikselowe, następnie rysuje trzy krawędzie trójkąta i wypełnia jego wnętrze metodą floodfill.

```
function Im = triangle(X, Y, A, B, C, dx, dy, color)
    % X, Y - rozmiar obrazu
    % A, B, C - wierzchołki trojkata
    % dx, dy - rozmiary pixela
    % color - kolor trojkata

    Im = uint8(zeros(Y, X));

    [x1,y1] = mat_to_pix(A(1), A(2), X, Y, dx, dy);
    [x2,y2] = mat_to_pix(B(1), B(2), X, Y, dx, dy);
    [x3,y3] = mat_to_pix(C(1), C(2), X, Y, dx, dy);

    Im = line(Im, y1, x1, y2, x2, dx, dy, color); % rysowanie krawedzi AB
    Im = line(Im, y2, x2, y3, x3, dx, dy, color); % rysowanie krawedzi BC
    Im = line(Im, y3, x3, y1, x1, dx, dy, color); % rysowanie krawedzi CA

    % znalezienie punktu w srodku trojkata
```

```
x_mid = round((x1 + x2 + x3)/3);  
y_mid = round((y1 + y2 + y3)/3);  
  
Im = floodfill(Im, y_mid, x_mid, 0, color); % wypelnianie trojkata  
end
```

Funkcja rysuje odcinek na obrazie monochromatycznym.

```
function Im = line(Im, ya, xa, yb, xb, dx, dy, color)  
% Im - obraz  
% (xa, ya) - punkt poczatkowy  
% (xb, yb) - punkt koncowy  
% color - kolor linii  
  
[Y, X] = size(Im);  
  
if (abs(yb-ya) > abs(xb-xa)) % stroma linia  
    % zamiana wspolrzednych  
    x0 = ya;  
    y0 = xa;  
    x1 = yb;  
    y1 = xb;  
  
    % zamiana rozmiarow X i Y  
    temp = X; X = Y; Y = temp;  
  
    zamiana = 1;  
else  
    x0 = xa;  
    y0 = ya;  
    x1 = xb;  
    y1 = yb;  
  
    zamiana = 0;  
end  
  
% zamiana poczatku i konca linii, zeby zaczynac od lewej  
if(x0 > x1)  
    temp1 = x0; x0 = x1; x1 = temp1;  
    temp2 = y0; y0 = y1; y1 = temp2;  
end  
  
dx = abs(x1 - x0) ; % odleglosc x  
dy = abs(y1 - y0); % odleglosc y  
sy = sign(y1 - y0); % znak przyrostu w kierunku y  
  
x = x0; y = y0; % inicjalizacja  
if x > 0 && x <= X && y > 0 && y <= Y  
    if (zamiana == 0)  
        Im(y,x) = color; % rysowanie punktu  
    else
```

```
        Im(x,y) = color;
    end
end

error = 2*dy - dx ;                % inicjalizacja bledu
for i = 0:dx-1

    error = error + 2*dy;           % modyfikacja bledu
    if (error > 0)
        y = y + sy;
        error = error - 2*dx;
    end

    x = x + 1;                      % zwiekszamy x
    if x > 0 && x <= X && y > 0 && y <= Y
        if (zamiana == 0)
            Im(y,x) = color;       % rysowanie punktu
        else
            Im(x,y) = color;
        end
    end
end
end
end
```

Funkcja wypelnia zamkniety obszar obrazu metoda floodfill.

```
function Im = floodfill(Im, y, x, T, color)
% Im - obraz
% (y,x) - punkt startowy
% T - jasnoc do zamiany
% color - nowy kolor

[Y, X] = size(Im);

% sprawdzenie zakresu
if y < 1 || y > Y || x < 1 || x > X
    error('Punkt startowy (y,x) jest poza obrazem.');
```

trójkąt jest obrucony do nas bokiem.

```
end

% sprawdzenie jasnosci piksela startowego
if Im(y,x) ~= T
    %disp('Piksel startowy ma zla jasnoc.');
```

trójkąt jest obrucony do nas bokiem.

```
    return;
    % w naszym kodzie oznacza to że trafiliśmy na krawędź - a więc
    % jesli nowa jasnoc taka sama jak stara, nic nie trzeba robic
    if T == color
        return;
    end
end
```

```
% kolejka pikseli do odwiedzenia
Q = zeros(Y*X, 2);
head = 1;
tail = 1;

Q(tail,:) = [y, x];
Im(y,x) = color;

while head <= tail
    cy = Q(head,1);
    cx = Q(head,2);
    head = head + 1;

    % 4-sasiedztwo: gora, dol, lewo, prawo

    % gora
    ny = cy - 1;
    nx = cx;
    if ny >= 1 && Im(ny,nx) == T
        tail = tail + 1;
        Q(tail,:) = [ny, nx];
        Im(ny,nx) = color;
    end

    % dol
    ny = cy + 1;
    nx = cx;
    if ny <= Y && Im(ny,nx) == T
        tail = tail + 1;
        Q(tail,:) = [ny, nx];
        Im(ny,nx) = color;
    end

    % lewo
    ny = cy;
    nx = cx - 1;
    if nx >= 1 && Im(ny,nx) == T
        tail = tail + 1;
        Q(tail,:) = [ny, nx];
        Im(ny,nx) = color;
    end

    % prawo
    ny = cy;
    nx = cx + 1;
    if nx <= X && Im(ny,nx) == T
        tail = tail + 1;
        Q(tail,:) = [ny, nx];
        Im(ny,nx) = color;
    end
end
end
```

Funkcja do zamiany współrzędnych matematycznych na pikselowe i odwrotnie

x, y - współrzędne M, N - szerokość i wysokość obrazu dx, dy - rozmiar pojedynczego piksela

```
function [i,j] = mat_to_pix(x, y, M, N, dx, dy)
    T = [1/dx, 0, M/2;
         0, -1/dy, N/2;
         0, 0, 1];
    temp = [x; y; 1];
    res = T * temp;
    i = round(res(1)/res(3));
    j = round(res(2)/res(3));
end
```

```
function [i,j] = pix_to_mat(x, y, M, N, dx, dy)
    T = [dx, 0, -M*dx/2;
         0, -dy, N*dy/2;
         0, 0, 1];
    res = T * [x; y; 1];
    i = res(1)/res(3);
    j = res(2)/res(3);
end
```

Funkcja do wyznaczania parametrów płaszczyzny $Ax+By+Cz+D=0$ przechodzącej przez trzy punkty p1,p2,p3

```
function [A, B, C, D] = plane(p1,p2,p3)
    A = det([p1(2),p1(3),1;
             p2(2),p2(3),1;
             p3(2),p3(3),1]);
    B = -det([p1(1),p1(3),1;
              p2(1),p2(3),1;
              p3(1),p3(3),1]);
    C = det([p1(1),p1(2),1;
             p2(1),p2(2),1;
             p3(1),p3(2),1]);
    D = -det([p1(1),p1(2),p1(3);
              p2(1),p2(2),p2(3);
              p3(1),p3(2),p3(3)]);
end
```

Funkcja do wyznaczania odleglosci punktu [x,y,z] od plaszczyzny o rownaniu $Ax+By+Cz+D=0$

odleglosc mierzona jest wzdluz prostej o wektorze kierunkowym [l,m,n] zwraca rowniez punkt przeciecia z plaszczyzna

```
function [d,x1,y1,z1] = dist_to_plane(plane, x, y, z, l, m, n)
    ro = (plane(1)*x+plane(2)*y+plane(3)*z+plane(4))/(
plane(1)*l+plane(2)*m+plane(3)*n);
    x1 = x-l*ro; y1 = y-m*ro; z1 = z-n*ro;
    d = sqrt((x-x1)^2+(y-y1)^2+(z-z1)^2);
end
```

Funkcja tworzy macierz transformacji jednorodnej 4x4 dla obrotu i przesuniecie.

Obrót jest złożeniem trzech obrotów: roll, pitch i yaw. Następnie dodawane jest przesunięcie o wektor [px, py, pz].

```
function Hmat = RPYT(roll,pitch,yaw,px,py,pz)
    % roll - obrót wokół osi OZ, w stopniach
    % pitch - obrót wokół osi OY, w stopniach
    % yaw - obrót wokół osi OX, w stopniach
    % px, py, pz - współrzędne przesunięcia
    r = roll*pi/180; p = pitch*pi/180; y = yaw*pi/180;
    MR = [cos(r), -sin(r), 0;
          sin(r), cos(r), 0;
          0, 0, 1];
    MP = [cos(p), 0, sin(p);
          0, 1, 0;
          -sin(p), 0, cos(p)];
    MY = [1, 0, 0;
          0, cos(y), -sin(y);
          0, sin(y), cos(y)];
    Rot = MY*MP*MR;
    Hmat = zeros(4,4);
    Hmat(1:3,1:3)=Rot;
    Hmat(1,4)=px; Hmat(2,4)=py; Hmat(3,4)=pz;
    Hmat(4,4)=1;
end
```

Published with MATLAB® R2026a